Práctica. Pon en marcha tu web y sácale los secretos de dentro

Nombre y apellidos: _____

Martes 17 de noviembre de 2026 · PPS · 4 horas · trabajo individual

Hoy empieza el proyecto que nos va a acompañar hasta diciembre. Te doy una pequeña aplicación web de verdad —reservas del aula de informática, hecha con Flask y una base de datos— y a partir de aquí es tuya. Hoy en Puesta en Producción Segura la pones en marcha como se pone en producción (con un servidor de aplicaciones y nginx delante) y le arreglas lo primero que se arregla: los secretos metidos en el código. El jueves, en Hacking Ético, atacarás **esta misma web** desde tu Kali. Y los miércoles, en Incidentes, tu Wazuh vigilará **estos mismos registros**. Una sola web, tres asignaturas.

La aplicación que te doy funciona, pero está escrita como se escribe el primer día: con la contraseña del administrador y la clave de sesión puestas a mano dentro de un fichero `.py`. Eso es lo más común y lo más peligroso del mundo real: un secreto en el código acaba en el repositorio, en la copia de seguridad y en la pantalla de quien no debe. Tu trabajo de hoy es ponerla en marcha y sacar esos secretos fuera, al entorno, sin romper nada. Al final tienes que pasar unas pruebas que hoy fallan a propósito.

1. Qué vas a aprender hoy

Al terminar la sesión tienes que ser capaz de:

1. Preparar y arrancar una aplicación Flask en un entorno virtual.
2. Ponerla en producción con gunicorn y nginx por delante.
3. Explicar por qué no se guardan secretos ni configuración en el código.
4. Leer la configuración desde variables de entorno y un fichero `.env` que no se sube al repositorio.
5. Generar una clave de sesión fuerte y guardarla donde toca.
6. Registrar eventos sin filtrar datos sensibles.
7. Auditar las dependencias de un proyecto con `pip-audit` y entender qué te dice.

2. Lo que necesitas

Conocimientos: lo de las prácticas anteriores (entorno virtual, pytest, Bandit). Hoy no hace falta Docker ni Git más allá de un commit final.

Equipo: tu Ubuntu 26.04 del aula (Python 3.14). Trabaja en el Ubuntu, no en la Kali. Para la parte de nginx y el servicio necesitarás `sudo`; si en tu puesto no lo tienes, avísame y hacemos esa parte juntos.

Material: descarga `PPS-2026-11-17-miweb-LAB.zip` de la web del curso (PPS, Prácticas) y descomprímelo.

```
mkdir -p ~/pps
cd ~/pps
unzip ~/Descargas/PPS-2026-11-17-miweb-LAB.zip
cd PPS-2026-11-17-miweb-LAB
ls
```

Tienes que ver la carpeta `miweb` (la aplicación), `tests`, `despliegue`, `wsgi.py` y `requirements.txt`. Abre el `README.md` y léelo: te cuenta qué es cada cosa.

3. Arranca la web en modo desarrollo (25 minutos)

Primero la ponemos en marcha de la forma más sencilla, para verla funcionar. Entorno virtual e instalación:

```
python3 -m venv .venv
. .venv/bin/activate
pip install -r requirements.txt
```

Comprueba que las pruebas que vienen de serie pasan (las que dicen que la web hace lo que tiene que hacer):

```
python -m pytest tests/test_funciona.py
```

Tienen que pasar todas. Ahora arráncala en modo desarrollo. Como la aplicación necesita dos secretos para funcionar y todavía no sabe leerlos del entorno, de momento los coge de dentro del código; eso es justo lo que vamos a cambiar:

```
. .venv/bin/activate
python -c "from miweb import crear_app; crear_app().run(port=8000)"
```

Abre el navegador del Ubuntu en `http://127.0.0.1:8000/`. Entra con el usuario `lucia` y la contraseña `Lucia.2026` (son datos de ejemplo que trae la base de datos). Date una vuelta: crea una reserva, mira el listado. Cuando lo tengas visto, para el servidor con Control+C.

4. El problema: secretos en el código (20 minutos)

Abre `miweb/config.py`. Mira estas dos líneas:

```
SECRET_KEY = "reservas-2026-cambiar"
ADMIN_CLAVE = "Admin2026!"
```

La primera es la clave con la que Flask firma la cookie de sesión: quien la conozca puede falsificar la sesión de cualquier usuario, incluido el administrador. La segunda es la contraseña del administrador. Las dos están escritas dentro de un fichero que, en un proyecto real, va al repositorio y lo ve todo el equipo (y cualquiera que acceda al repositorio).

En la terminal, comprueba cómo de fácil es encontrarlas:

```
grep -rn "SECRET_KEY\|ADMIN_CLAVE\|Admin2026" miweb/
```

Esto es exactamente lo que hace una herramienta de análisis de secretos (y un atacante) en cuanto pone la mano en el código. El arreglo no es "esconderlas mejor": es **sacarlas del código** y que la aplicación las lea de su entorno, que es de donde se leen los secretos en producción.

5. Configuración por entorno y fichero `.env` (45 minutos)

La idea: el código no lleva valores, lleva *de dónde* sacar los valores. Los valores de verdad viven en variables de entorno, y para desarrollo se cargan desde un fichero `.env` que **nunca** se sube al repositorio.

5.1. Cambia `config.py`

Reescribe `miweb/config.py` para que cada ajuste se lea del entorno. Usa `os.environ`. Te dejo cómo tiene que quedar la cabecera y la primera variable; haz igual con el resto (`ADMIN_CLAVE`, `DEBUG`, `BD`, `REGISTRO_SEGURIDAD`):

```
import os
from pathlib import Path

RAIZ = Path(__file__).resolve().parent.parent

SECRET_KEY = os.environ.get("MIWEB_SECRET_KEY")
```

Fíjate en dos cosas. Una: `os.environ.get(...)` devuelve `None` si la variable no está, y eso es lo que queremos (si falta el secreto, la aplicación no debe arrancar con uno por defecto). Dos: `DEBUG` tiene que ser un booleano; una variable de entorno siempre es texto, así que compáralo (por ejemplo, que valga `True` solo cuando `MIWEB_DEBUG` sea `"1"`). Por defecto, con el modo depuración **apagado**: en producción, el modo depuración enseña el código y los datos cuando hay un error.

5.2. Que no arranque sin secretos

Abre `miweb/__init__.py` y busca la función `crear_app`. Justo después de cargar la configuración, añade una comprobación: si falta `SECRET_KEY` o `ADMIN_CLAVE`, que la aplicación **no arranque** y lo diga claro. Así nadie la sube a producción sin configurarla:

```
    for obligatoria in ("SECRET_KEY", "ADMIN_CLAVE"):
        if not app.config.get(obligatoria):
            raise RuntimeError(f"Falta la variable de entorno
MIWEB_{obligatoria}")
```

5.3. Crea tu `.env`

En `despliegue/` tienes `miweb.env.ejemplo`. Es la **plantilla**: dice qué variables hay, pero sin valores. Cópiala a tu carpeta de configuración personal y rellénala:

```
mkdir -p ~/.config/miweb
cp despliegue/miweb.env.ejemplo ~/.config/miweb/miweb.env
```

Genera una clave de sesión de verdad (aleatoria y larga), no te la inventes tú:

```
python3 -c "import secrets; print(secrets.token_hex(32))"
```

Edita `~/.config/miweb/miweb.env` con `nano`: pega esa clave en `MIWEB_SECRET_KEY`, pon una contraseña de administrador de 12 caracteres o más en `MIWEB_ADMIN_CLAVE`, deja `MIWEB_DEBUG=0` y ajusta las rutas a tu usuario. Protégelo para que solo tú lo leas:

```
chmod 600 ~/.config/miweb/miweb.env
```

Comprueba que el `.gitignore` del proyecto ya ignora los `.env` (para que no se te cuele en un commit):

```
grep env .gitignore
```

5.4. Prueba que ya lee del entorno

Carga tu `.env` en la terminal y arranca:

```
set -a; . ~/.config/miweb/miweb.env; set +a
python -c "from miweb import crear_app; crear_app().run(port=8000)"
```

`set -a` hace que todo lo que se defina a continuación se exporte como variable de entorno; así la aplicación ve tu `.env`. Entra como `admin` con la contraseña que acabas de poner. Si arranca y entras, ya lee del entorno. Para el servidor.

Y comprueba que si quitas un secreto, **no arranca** (esto es lo que queremos):

```
env -u MIWEB_SECRET_KEY python -c "from miweb import crear_app; crear_app()"
```

Tiene que terminar con el `RuntimeError` que has puesto. Si arranca igual, es que la comprobación del 5.2 no está bien.

6. Registrar sin filtrar (30 minutos)

Abre `miweb/registro.py` y mira la función `evento`: escribe una línea JSON por cada cosa que pasa (login correcto, fallido, acceso denegado...). Ese fichero es el que leerá tu Wazuh en Incidentes, así que los nombres de los campos no se tocan.

Pero hay un problema. Busca en el código dónde se registra un login fallido:

```
grep -n "login_fallido" miweb/__init__.py
```

Verás que, cuando la contraseña es incorrecta, el detalle que se guarda **incluye la contraseña que se ha intentado**. Eso es un fallo de los gordos: el registro de seguridad acaba siendo una lista de contraseñas en claro, y encima la que el usuario de verdad ha tecleado mal (que a lo mejor es su contraseña de otro sitio). Arréglalo: que el detalle diga que la contraseña era incorrecta, **sin** escribir cuál era.

Un registro de seguridad apunta *qué* ha pasado y *quién* y *desde dónde*, nunca el secreto en sí. Lo mismo vale para tokens, números de tarjeta o datos personales de más.

Comprueba que lo has arreglado:

```
set -a; . ~/.config/miweb/miweb.env; set +a
python -m pytest tests/test_secretos.py
```

Estas son las pruebas que hoy fallaban a propósito. Cuando hayas hecho los apartados 5 y 6, **tienen que pasar todas**. Si alguna sigue roja, lee qué comprueba y vuelve sobre ese punto.

7. Ponla en producción: gunicorn y nginx (40 minutos)

El `app.run()` que hemos usado es el servidor de desarrollo de Flask: va bien para probar, pero no aguanta producción y él mismo te avisa. En producción se usa un **servidor de aplicaciones** (gunicorn) y, por delante, un **servidor web** (nginx) que atiende a la red.

Instala gunicorn (ya está en `requirements.txt`, compruébalo) y lanza la web con él para verlo:

```
set -a; . ~/.config/miweb/miweb.env; set +a
.venv/bin/gunicorn --workers 2 --bind 127.0.0.1:8000 wsgi:app
```

`wsgi:app` le dice a gunicorn que la aplicación está en `wsgi.py`, en la variable `app`. Fíjate en que escucha en `127.0.0.1`: solo tu propio equipo, no la red. Para quien atienda a la red está nginx. Para el servidor con Control+C.

Ahora lo dejamos montado como servicio, que arranque solo y con nginx delante. Te he preparado un script que hace todo eso (crea el servicio, configura nginx, comprueba que responde):

```
sh despliegue/instalar-servicio.sh
```

Léelo antes de ejecutarlo (`cat despliegue/instalar-servicio.sh`): no me creas porque sí, mira qué hace. Te pedirá la contraseña de `sudo`. Si termina diciendo que miweb responde en `http://127.0.0.1:8080/`, ya está en marcha de verdad.

Mira la configuración de nginx que instala, en `despliegue/nginx-miweb.conf`. Dos detalles importantes para las otras asignaturas:

- Escucha en el puerto **8080** y solo atiende a `127.0.0.1` y a la red `192.168.56.0/24` (tu Kali); al resto le responde 403. Tu web tiene fallos a propósito y no puede quedar abierta a toda la red del aula.
- Le pasa a la aplicación la IP real del cliente con `X-Forwarded-For`, poniéndola él (no dejando que la mande el cliente). Por eso en los registros verás la IP de quien entra, que es lo que necesita Wazuh.

8. Conecta tu Kali para el jueves (20 minutos)

El jueves atacarás esta web desde tu Kali. Para que en los registros aparezca la IP de la Kali (y no `127.0.0.1`), las dos máquinas tienen que verse por una red aparte. Te he preparado otro script; con la **Kali apagada**, desde el Ubuntu:

```
bash despliegue/red-solo-anfitrion.sh
```

Crea una red solo-anfitrión de VirtualBox: tu Ubuntu será `192.168.56.1` y tu Kali cogerá una IP `192.168.56.1xx`. No toca la tarjeta que la Kali ya usa para `labnet` ni la de Internet. Cuando termine, arranca la Kali y comprueba desde ella:

```
ip -br a
curl -sI http://192.168.56.1:8080/
```

Tienes que ver una IP `192.168.56.x` en la Kali y que el `curl` responde (un `HTTP/1.1 200` o un `302`). Si responde, el jueves empiezas directo. Si no, avísame; no toques la red a mano.

9. Audita las dependencias (25 minutos)

Tu web usa Flask y unas cuantas bibliotecas más. Cualquiera de ellas puede tener una vulnerabilidad conocida, y entonces tu web la hereda aunque tu código esté perfecto. `pip-audit` compara lo que tienes instalado con una base de datos de vulnerabilidades.

```
pip install pip-audit
pip-audit -r requirements.txt
```

Lee la salida: para cada paquete con problemas te da el identificador de la vulnerabilidad y la versión donde está corregida. Vas a ver que alguna de las dependencias que te he dado tiene una versión con un fallo conocido y una versión que lo arregla.

Arréglalo subiendo esa dependencia a la versión corregida **en `requirements.txt`** (edita el número de versión), reinstala y vuelve a auditar:

```
pip install -r requirements.txt
pip-audit -r requirements.txt
```

Repite hasta que `pip-audit` no se queje, o hasta que el único aviso que quede sea uno que no tenga todavía versión con arreglo (en ese caso, apúntalo: no siempre se puede corregir en el momento, pero hay que saberlo). Comprueba que la web sigue pasando sus pruebas después de subir las versiones:

```
python -m pytest
```

10. Preguntas

1. ¿Por qué es peligroso guardar un secreto en el código aunque el repositorio sea privado?
2. ¿Qué diferencia hay entre el fichero `.env` y el `miweb.env.ejemplo`? ¿Cuál se sube al repositorio y cuál no, y por qué?
3. ¿Por qué hacemos que la aplicación no arranque si falta un secreto, en vez de ponerle uno por defecto?
4. ¿Qué dato estaba filtrando el registro de seguridad y por qué es grave?
5. ¿Por qué gunicorn escucha en `127.0.0.1` y no en la red, si quien tiene que atender a la red es nginx?
6. De lo que te ha dicho `pip-audit`, ¿qué has corregido y qué no has podido, y por qué?

11. Si algo no funciona

`pip install` falla con un error de compilación: comprueba que estás en el entorno virtual (el prompt tiene `(.venv)`). Si no, actívalo con `. .venv/bin/activate`.

La web no arranca y dice `Falta la variable de entorno MIWEB_...`: no has cargado tu `.env`. Haz `set -a; . ~/.config/miweb/miweb.env; set +a` en esa misma terminal antes de arrancar. Recuerda que las variables solo valen en la terminal donde las cargas; si abres otra, vuelve a cargarlas.

`instalar-servicio.sh` se para diciendo que falta el `.env` o que tiene permisos mal: crea `~/.config/miweb/miweb.env` (apartado 5.3) y dale permisos con `chmod 600`.

`sudo nginx -t` da error: casi siempre es que otro servicio ya usa el puerto 8080, o que has editado el fichero de nginx y falta un `;` o una llave. Lee la línea que te dice y corrígela.

El servicio no levanta: mira el motivo con `sudo journalctl -u miweb -n 30 --no-pager`. Lo más común es que el `.env` tenga una ruta que no existe; créala o corrígela.

`red-solo-anfitrion.sh` dice que la Kali no está apagada: apágala del todo (no vale suspenderla) y vuelve a lanzarlo.

12. Qué tienes que entregar (20 minutos)

Carpeta `NN-PPS-miweb` en ZIP con:

1. Tu `miweb/config.py` y `miweb/__init__.py` ya arreglados (sin secretos dentro).
2. La salida de `python -m pytest` con todo en verde (cópiala a un fichero de texto).
3. La salida de `pip-audit` **antes** y **después** de arreglar las dependencias.
4. El `requirements.txt` con las versiones corregidas.
5. Las respuestas a las preguntas del apartado 10.

No entregues tu `~/.config/miweb/miweb.env`: ese fichero tiene tus secretos y no sale de tu equipo. Si lo encuentro en una entrega, esa parte cuenta como suspendida: es justo el error que hoy aprendemos a no cometer.

13. Antes de entregar, comprueba que

1. No queda ningún secreto escrito en los ficheros `.py` (pásale el `grep` del apartado 4).
2. La aplicación no arranca si falta un secreto.
3. El registro de seguridad ya no guarda la contraseña de los intentos fallidos.
4. `python -m pytest` pasa entero (las de funcionamiento y las de secretos).
5. Tu `.env` no está en la entrega ni subido a ningún repositorio.

14. Seguridad y por qué importa

Un secreto en el código es la primera causa de incidente que vas a ver en tu vida profesional: alguien sube una clave a un repositorio, un robot la encuentra en minutos y entra. Sacarla al entorno, generar claves fuertes y no dejar que la aplicación arranque mal configurada es de lo más barato y lo más efectivo que puedes hacer por una web. Lo que montas hoy lo vas a seguir

trabajando: el lunes que viene la bastionas (cabeceras, cookies, CSP) y entre medias, en Hacking Ético e Incidentes, la atacas y la vigilas. Guárdala bien.

Referencias

- Las doce reglas de una aplicación moderna, configuración por entorno: https://12factor.net/es/config
- Flask, configuración desde variables de entorno: https://flask.palletsprojects.com/en/stable/config/
- gunicorn, puesta en marcha: https://docs.gunicorn.org/en/stable/run.html
- nginx como proxy inverso: https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy/
- pip-audit: https://pypi.org/project/pip-audit/
- OWASP, exposición de datos sensibles y mala configuración: https://owasp.org/Top10/